ReleaseShield — Complete Technical Document

From Raw Scanner Output to a Unified Security CLI

PAGE 1 — The Core Problem: Two Tools, Same Bug, No Connection

---

What Is the Problem, Simply Put?

When your team runs security scans on a web application, you use two very different kinds of tools:

- **DAST tools** (like OWASP ZAP) act like a hacker. They visit your live website and poke it with bad inputs. They see **URLs**.
- **SAST tools** (like Semgrep) read your source code without running it. They find dangerous code patterns. They see **files and line numbers**.

Both tools may find the **exact same vulnerability**, but they describe it in completely different languages. There is no automatic link between them.

A Real Example

Imagine your app has a SQL Injection bug in a search feature.

ZAP finds it at runtime (black-box view):

```
Alert:  SQL Injection
URL:    https://app.example.com/api/v1/search?q=test
Param:  q
```

Semgrep finds it in the source code (white-box view):

```
Rule:   python.django.security.injection.sql
File:   controllers/search.py  (line 42)
Code:   cursor.execute("SELECT * FROM items WHERE name =
```

These are **the same bug**. But if you look at both reports side by side, **nothing matches**. ZAP does not know the file name. Semgrep does not know the URL. Without extra work, you cannot tell a developer "this ZAP alert and this Semgrep finding are the same problem — fix it once."

Why Is Connecting Them Hard?

The missing piece is a **route-to-handler map** — a lookup table that says:

> "The URL `/api/v1/search` is handled by the function `search_handler()` in the file `controllers/search.py`."

Building this map is hard for four reasons:

Reason	Plain English Explanation
Routes are defined elsewhere	The URL `/api/v1/search` is written in `urls.py` or as a decorator `@app.route(...)`, not in the handler file itself
The bug may be in a helper	`search_handler()` may call `query_builder.py`, where the actual dangerous code lives

Prefixes are added by routers	`/api/v1` may be added by a "blueprint" or "router" in a completely different file
Dynamic routes don't match literally	ZAP visits `/users/17` but the route is defined as `/users/<id>` — they don't match as strings

---

What the Map Looks Like

```
URL Pattern              Method   Handler File
/api/v1/search           GET      controllers/search.py
/api/v1/users/{id}       GET      controllers/users.py
/api/v1/login            POST     controllers/auth.py
```

With this map, the correlation becomes a simple 4-step join:

1. **Normalize** the ZAP URL → strip the host, query string, and replace real IDs with `{id}` → `https://app.example.com/api/v1/search?q=test` → `/api/v1/search`
2. **Look up** the handler in the map → `controllers/search.py :: search_handler`
3. **Match** Semgrep findings in that file and function (and any functions it calls)
4. **Mark confirmed** → "This static flaw has runtime proof of exploitability"

Four Ways to Build the Map (With Trade-offs)

Approach	How It Works	Trade-off
Parse OpenAPI/ Swagger spec	Read the `paths:` section and `operationId`	Only works if spec is up to date and linked to code

Framework route introspection	Ask Flask for `app.url_map`, Django for `show_urls`, Spring for `/actuator/mappings`	Very accurate, but different code per framework
Static route extraction	Use Semgrep/AST rules to find `@app.route(...)` decorators	Language-specific, misses routes built at runtime
Heuristics	Match URL segments to file or function names by convention	Fast to build, but often wrong

> **For a hackathon:** OpenAPI parsing or Flask/Django route introspection is the most practical choice. Pick one framework and do it well.

PAGE 2 — The Storage Layer: SARIF + PostgreSQL

What Is SARIF?

SARIF (Static Analysis Results Interchange Format) is a standard JSON format invented so that every security tool speaks the same language. Instead of parsing Trivy's custom JSON, then Semgrep's custom JSON, then Gitleaks's custom JSON — you parse **one structure** for all of them.

Most modern tools already output SARIF directly:

- `semgrep --sarif > semgrep.sarif`
- `trivy image --format sarif > trivy.sarif`
- `gitleaks detect --report-format sarif > gitleaks.sarif`

> **Important correction:** You do NOT need to write code to convert tool output to SARIF. These tools do it themselves. Your code only needs to **validate and merge** the SARIF files they produce.

What Is a SARIF File? (Simplified Structure)

```
{
  "version": "2.1.0",
  "runs": [
    {
      "tool": { "driver": { "name": "Semgrep" } },
      "results": [
        {
          "ruleId": "python.django.security.injection.sql
          "level": "error",
          "locations": [{
            "physicalLocation": {
              "artifactLocation": { "uri": "controllers/s
              "region": { "startLine": 42 }
            }
          }],
          "fingerprints": { "primaryLocationLineHash": "a
        }
      ]
    }
  ]
}
```

Every result has: a rule ID, a severity level, a file location, and a fingerprint (a hash used to track the same finding across scans).

Why PostgreSQL with jsonb?

You store the raw SARIF file as-is in a Postgres `jsonb` column. This gives you two benefits at once:

1. **Keep everything** — the full original report is never lost
2. **Query it fast** — Postgres can index inside the JSON

Key indexes to add:

```
-- Find by rule
CREATE INDEX ON findings ((data->>'ruleId'));

-- Find by file and line
CREATE INDEX ON findings ((data->'locations'->0->'physica

-- De-duplicate across scans using fingerprint
CREATE INDEX ON findings ((data->'fingerprints'->>'primar
```

What SARIF + Postgres Does NOT Solve

SARIF normalizes the **shape** of findings, not the **content**. The problem remains:

```
Semgrep finding  →  file: controllers/search.py, line: 42
ZAP finding      →  url:  /api/v1/search
```

These two rows sit in the same Postgres table but have **nothing in common** to join on. SARIF doesn't fix that. The route map from Page 1 is still required.

To fully solve correlation, you need three tables:

```
findings      — raw SARIF results from all tools
routes        — URL pattern → file → function (built from
correlations  — links a ZAP finding ID to a Semgrep findi
```

And a **cross-tool fingerprint** for confirmed vulnerabilities:

```
cross_fingerprint = hash(vuln_class + handler_function +
```

> **Verdict:** SARIF + Postgres is an excellent storage and ingestion layer. It is not a complete answer to the correlation problem. It's the foundation you build on top of.

PAGE 3 — The CLI Design and 3-Layer Architecture

The Three-Layer Architecture (The Most Important Design Decision)

Think of your system like a restaurant:

```
┌──────────────────────────────────────────────────────┐
│  LAYER 1 — RUNNERS                                   │
│  The kitchen. Runs Trivy, Semgrep, Gitleaks.         │
│  Each runner calls the tool and collects its         │
│  SARIF output file. Nothing else.                    │
└──────────────────────────────────────────────────────┘
                          │  produces
                          ▼
┌──────────────────────────────────────────────────────┐
│  LAYER 2 — DATA / MERGE                              │
│  The chef's prep table. Merges all SARIF files       │
│  into one. Validates structure. Counts findings      │
│  by severity. Builds the summary object.             │
│  Knows nothing about how results will be shown.      │
└──────────────────────────────────────────────────────┘
                          │  produces
                          ▼
┌──────────────────────────────────────────────────────┐
│  LAYER 3 — DISPLAY                                   │
│  The waiter. Takes the summary data and presents     │
│  it. In Sprint 1: a CLI table. In Sprint 4:          │
```

```
|  a web dashboard. The same data, different face.    |
└─────────────────────────────────────────────────────┘
```

Why this matters: When you build the website in Sprint 4, you only write a new display layer. Layers 1 and 2 don't change at all. This is the correct way to design for growth.

The CLI — What It Shows and Why

The wrong thing to do is print all 70+ findings. That is exactly the noise your product is supposed to eliminate. The right output is a **progress view** then a **summary table**.

Correct CLI output:

```
$ releaseshield scan ./NodeGoat

Select tools:  [x] Trivy  [x] Semgrep  [x] Gitleaks

  Trivy      done    18.2s
  Semgrep    done    41.7s
  Gitleaks   done     6.9s

  Tool        High   Medium   Low   Total
  ──────────────────────────────────────────
  Trivy         12       20    10      42
  Semgrep       11       15     5      31
  Gitleaks       3        0     0       3
  ──────────────────────────────────────────
  Total         26       35    15      76     (total time

  Saved: ./results/result.sarif
```

If a tool fails: `Gitleaks failed (exit code 1: binary not found)` — and the rest still prints.

CLI Commands and Flags

```
releaseshield scan <repo>                    # Interactive
releaseshield scan <repo> --tools trivy      # Run only Tr
releaseshield scan <repo> --tools all        # Run all too
releaseshield scan <repo> --out result.sarif  # Custom ou
releaseshield scan <repo> --json             # Machine-rea
```

The `--json` flag outputs something like:

```
{
  "total_time_seconds": 42,
  "tools": {
    "trivy":   { "high": 12, "medium": 20, "low": 10, "to
    "semgrep": { "high": 11, "medium": 15, "low":  5, "to
    "gitleaks":{ "high":  3, "medium":  0, "low":  0, "to
  },
  "sarif_path": "./results/result.sarif"
}
```

This costs almost nothing to add but will be critical when the GitHub Action reads your output.

Things to Watch in Implementation

Issue	What to Do
Parallel tool timing	Each tool shows its own duration. Total = slowest tool, not sum
Severity mapping	SARIF uses `error/warning/note`. For Sprint 1, count those. Map to High/Medium/Low in Sprint 2
One tool failing	Catch it, show the error inline, continue with the others
SARIF merge	Combine multiple `runs[]` arrays into one SARIF file

Recommended Libraries:

Language	Menu + Args	Tables, Colors, Progress
Python	Typer + questionary	Rich
TypeScript	Commander + Inquirer	chalk + cli-table3 + ora

PAGE 4 — Sprint 1: Complete Details

Sprint 1 Goal (One Sentence)

> Run Trivy, Semgrep, and Gitleaks on a target repository in parallel, collect their SARIF output, merge it, and display a clean summary table in the CLI — all within 60 seconds.

Sprint 1 Acceptance Criteria (Definition of Done)

A Sprint 1 is complete when ALL of the following pass:

- ☐ `releaseshield scan ./NodeGoat` runs without crashing
- ☐ All three tools run and produce `.sarif` files
- ☐ Tools run in parallel (not one after another)
- ☐ Total scan time is under 60 seconds on NodeGoat
- ☐ The CLI shows live progress (tool name + elapsed time)
- ☐ The CLI shows the final summary table with High / Medium / Low / Total per tool
- ☐ The merged SARIF file is saved to `./results/result.sarif`
- ☐ `--tools` flag works (e.g. `--tools trivy`, `--tools all`)
- ☐ `--out` flag overrides the default output path
- ☐ `--json` flag prints a machine-readable summary to stdout
- ☐ If one tool fails, the others continue and the failure is shown

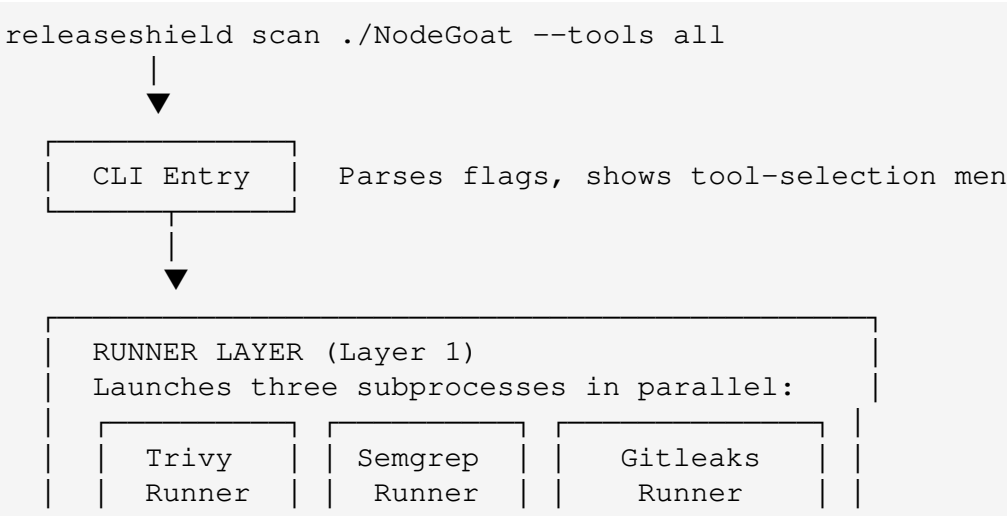
clearly
- ☐ README explains installation, usage, and the Sprint 1 severity mapping note

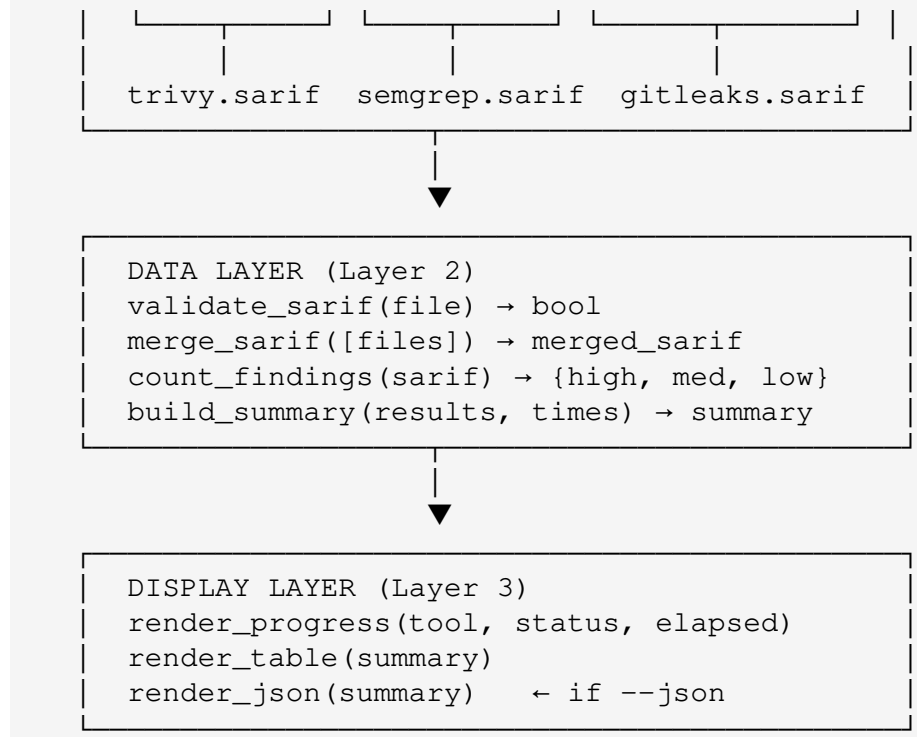
---

Sprint 1 Scope — What Is IN and What Is OUT

IN (Must Have)	OUT (Later Sprints)
Trivy, Semgrep, Gitleaks runners	ZAP / DAST integration
Parallel execution	PostgreSQL storage
SARIF merge (no conversion needed)	Route-to-handler map
CLI progress + summary table	Web dashboard
`--tools`, `--out`, `--json` flags	Cross-tool correlation
Error handling per tool	Severity re-mapping
NodeGoat as the test target	IDE extension

Sprint 1 Architecture in Detail





Sprint 1 File Structure

```
releaseshield/
├── cli/
│   ├── main.py          ← Entry point, argument parsing,
│   └── display.py       ← All Rich/chalk rendering, tabl
├── runners/
│   ├── base.py          ← Abstract Runner class
│   ├── trivy.py         ← Builds + runs Trivy command, r
│   ├── semgrep.py       ← Builds + runs Semgrep command,
│   └── gitleaks.py      ← Builds + runs Gitleaks command
├── core/
│   ├── merge.py         ← Merges multiple SARIF files in
│   ├── validate.py      ← Checks SARIF structure is vali
│   └── summary.py       ← Counts findings, builds summar
```

```
├── results/              ← Default output directory (git-
├── tests/
│   ├── test_merge.py
│   ├── test_summary.py
│   └── test_runners.py
├── README.md
└── requirements.txt      ← (or package.json for Node)
```

PAGE 5 — Sprint 1 Role Division (12 People)

Team Structure Overview

The 12 roles are divided into four functional groups. Every role has a clear owner and deliverable so that work happens in parallel, not in a queue.

Group A — Infrastructure & Setup (2 people)

Role 1 — Project Lead / Architect

- Sets up the repo, branch rules, CI workflow skeleton
- Defines the 3-layer architecture and code conventions
- Writes the base `Runner` class that all tool runners inherit
- Reviews all PRs for architectural correctness
- Owns the final integration: wires Layers 1, 2, 3 together
- **Deliverable:** Working repo scaffold + base runner class + CI pipeline

Role 2 — DevOps / Environment Setup

- Installs Trivy, Semgrep, Gitleaks in the CI environment (Docker or GitHub Actions)

- Downloads and prepares NodeGoat as the test target repository
- Writes the `Makefile` or `scripts/install.sh` so any team member can set up locally in one command
- Confirms all three tools run manually on NodeGoat and produce SARIF output
- **Deliverable:** Working local dev environment + CI environment + confirmed tool output samples

---

Group B — Runner Layer (3 people)

Each person owns one tool runner completely. They all inherit from the base Runner class defined by Role 1.

Role 3 — Trivy Runner

- Builds the exact Trivy command: `trivy fs --format sarif --output trivy.sarif <repo>`
- Handles Trivy exit codes (0 = success, 1 = findings found, >1 = error)
- Records start time and end time
- Returns: `{ status: "done"|"failed", sarif_path, elapsed_seconds, error_message }`
- Writes unit test with a mock SARIF output
- **Deliverable:** `runners/trivy.py` + tests

Role 4 — Semgrep Runner

- Builds the Semgrep command: `semgrep scan --sarif --output semgrep.sarif <repo>`
- Handles the case where no config/rules are found gracefully
- Records timing and result
- Writes unit test
- **Deliverable:** `runners/semgrep.py` + tests

Role 5 — Gitleaks Runner

- Builds the Gitleaks command: `gitleaks detect --source <repo> --report-format sarif --report-path gitleaks.sarif`

- Gitleaks exits with code 1 when secrets are found — this is NOT a failure, handle it correctly
- Records timing and result
- Writes unit test
- **Deliverable:** `runners/gitleaks.py` + tests

Group C — Data / Merge Layer (3 people)

Role 6 — SARIF Validator

- Reads the SARIF 2.1.0 spec (the schema is public JSON Schema)
- Writes `validate_sarif(filepath) → True/False + error list`
- Handles edge cases: empty `runs[]`, missing `results[]`, null `locations`
- Writes tests using the sample SARIF files produced by Role 2
- **Deliverable:** `core/validate.py` + tests

Role 7 — SARIF Merger

- Writes `merge_sarif(list_of_paths) → merged_sarif_dict`
- Logic: take all `runs[]` arrays from each file and concatenate them into one SARIF with multiple runs
- Writes the merged result to `./results/result.sarif`
- Handles partial failures: if one file is invalid, skip it and log a warning
- Writes tests with 2 and 3 input files
- **Deliverable:** `core/merge.py` + tests

Role 8 — Summary Builder

- Writes `count_findings(sarif_run) → {high, medium, low, total}`
- Sprint 1 mapping: SARIF `"error"` → High, `"warning"` → Medium, `"note"` → Low
- Writes `build_summary(runner_results, sarif_path) →`

`summary_dict`
- The summary dict is the contract between Layer 2 and Layer 3
- Documents the Sprint 1 severity simplification in code comments and README
- **Deliverable:** `core/summary.py` + tests + README note

Group D — CLI / Display Layer (4 people)

Role 9 — CLI Entry Point & Argument Parsing

- Writes `cli/main.py` using Typer (Python) or Commander (Node)
- Implements all flags: `--tools`, `--out`, `--json`
- Shows the interactive tool-selection menu when `--tools` is not passed (use questionary or Inquirer)
- Calls the parallel runner executor (coordinates with Role 11)
- Passes results to display functions
- **Deliverable:** `cli/main.py` fully wired

Role 10 — Display: Progress + Table

- Writes `cli/display.py` using Rich (Python) or chalk + cli-table3 (Node)
- `render_progress(tool_name, status, elapsed)` — live updating line per tool
- `render_table(summary_dict)` — the final High/Medium/Low/Total table
- `render_error(tool_name, error_message)` — failure line for a tool
- Matches the exact format shown in the spec (see Page 3 example)
- **Deliverable:** `cli/display.py` fully styled + tested with mock data

Role 11 — Parallel Execution Coordinator

- Writes the function that launches all selected runners in parallel
- Python: `asyncio.gather()` or

`concurrent.futures.ThreadPoolExecutor`
- Node: `Promise.all()`
- Feeds live progress updates to the display layer as each tool finishes
- Returns all results to the CLI entry point
- **Deliverable:** `core/executor.py` or equivalent

Role 12 — Integration Testing & README

- Writes the end-to-end integration test: runs `releaseshield scan ./NodeGoat --tools all` and asserts:
  - Exit code 0
  - `result.sarif` file exists and is valid SARIF
  - stdout contains the summary table
  - Total time < 60 seconds
- Also tests the `--json` flag output
- Writes the complete `README.md`: installation, usage, examples, the severity note, known limitations
- Acts as the "chaos monkey": intentionally breaks one tool binary and verifies the error handling works
- **Deliverable:** `tests/test_integration.py` + `README.md`

Sprint 1 Summary Table

Role	Name	Group	Key Deliverable
1	Project Lead	Infrastructure	Repo scaffold, base runner, architecture
2	DevOps	Infrastructure	CI setup, NodeGoat, tool verification
3	Trivy Runner	Runners	`runners/trivy.py` + tests
4	Semgrep Runner	Runners	`runners/semgrep.py` + tests
5	Gitleaks Runner	Runners	`runners/gitleaks.py` + tests

6	SARIF Validator	Data	`core/validate.py` + tests
7	SARIF Merger	Data	`core/merge.py` + tests
8	Summary Builder	Data	`core/summary.py` + tests
9	CLI Entry	Display	`cli/main.py` + flags + menu
10	Display Layer	Display	`cli/display.py` + progress + table
11	Parallel Executor	Display	`core/executor.py`
12	Integration & Docs	QA	E2E tests + README

Sprint 1 Timeline (Suggested 2-Week Sprint)

```
Week 1, Day 1–2:    Role 1 + 2 finish setup. Everyone els
Week 1, Day 2–5:    Roles 3–8 build runners and data laye
Week 2, Day 1–2:    Roles 9–11 wire the CLI. Roles 3–8 fi
Week 2, Day 3–4:    Role 12 runs integration tests. Bugs
Week 2, Day 5:      Demo: full scan on NodeGoat, under 60
```

> **What Sprint 2 adds:** Proper severity mapping (Critical/High from tool-specific SARIF fields), PostgreSQL storage, and the beginning of the route-to-handler map.
>
> **What Sprint 4 adds:** The web dashboard — a new display layer reading the same Layer 2 summary data. Layers 1 and 2 are untouched.